

Cognitive Interview Assessment

Computer Vision Data Pipeline Project

By: Kyle Webster

Introduction:

This computer vision project uses Python and Pytorch to implement an end-to-end image classification pipeline. Analyzing the Kaggle CatsAndDogs database of approximately 25,000 images of cats and dogs, to train on and images, classifying them as cats or dogs. The emphasis of this project is on code optimization and data flow. This project was developed with Git for version control.

Instructions:

Download source code from <https://github.com/elyk10/DogsVsCats>

Download the dataset from this link that was provided:

https://download.microsoft.com/download/3/e/1/3e1c3f21-ecdb-4869-8368-6deba77b919f/kagglecatsanddogs_5340.zip

Place the two folders in the dataset zip inside a folder called “/data” in the same directory that “/src” is inside from the GitHub download link.

Ensure Python is installed on computer.

For a Windows computer open the command prompt from folder holding “/src” and “/data” folder and run the following commands or run the “startup.bat” file. The following commands and the .bat file are used to set up a virtual environment, activate it and install the required dependencies.

```
python -m venv venv
.\venv\Scripts\activate
```

To use CPU for processing:

```
pip install torch torchvision matplotlib numpy tqdm pillow
```

If you have a GPU you can use the following commands to use it for processing:

```
pip install torch torchvision --index-url
https://download.pytorch.org/whl/cu126
```

```
pip install matplotlib numpy tqdm pillow
```

Once all the above is set up, you can run the code with the following command from the same directory

```
python src\main.py
```

Running:

The program requires no interaction while running other than closing windows from the matplotlib library to continue. The windows display an example image from the dataset after it has been transformed for the data loader and charts showing the training and validation accuracy and loss over epochs.

The model will be saved in a directory path of “\output\model” from where the code was run. A csv file containing a list of predicted classifications for images and a small group of actual images in jpg format named with prediction from the dataset will be saved in “\output\predictions”.

Description:

The model used for this computer vision project was ResNet-18, originally using the default weights, but after training using the save weights to make predictions. The classifier layer of the model was changed to two output features as the project was to classify two different class, Cats and Dogs. ResNet-18 was chosen as to a different variant of ResNet or a model like VGG16 due to improved speed on smaller data sets over the other versions and improved memory usage. It was also chosen as opposed to building a Convolutional Neural Network (CNN) from scratch to save development time.

Transformations that were done to dataset images include, resizing, cropping to center and normalizing the images after they were converted to tensors. They were all resized to be 256x256 and then the crop placed them at 224x224. This was done for all train, validation and test.

The model was trained using a cross-entropy loss function and optimized with a learning rate of 0.001 and momentum value of 0.9 for safe and stable training. 0.001 is a safe value without loss jumping around too much while going much lower would have slowed down processing times. 0.9 is also a safe common value for momentum. It maintains smooth updates and does not overshoot the loss function.

The data loaders were setup with batch sizes of 64 and number of workers set to 2. This was done to not slow down training with large batch sizes and to have training done on multiple threads by setting the number of workers to two. This lead to quicker processing times during training as opposed to using larger batches and less workers.

Loss and Accuracy values were chosen to keep track of effectiveness of model over epochs as they are the most effective to see how model is performing. They demonstrate the how wrong the predictions of the model were and how often they are correct. Low loss and high accuracy indicate the model is performing well. As shown in the results below the model trained in the project received a loss of <4% and an accuracy of >99%.

Design decisions for this project chose to prioritize speed and simplicity over 100% accuracy, due to time constraints to create project and limited hardware capabilities. Goal of >90% accuracy of model was still achieved with model training.

Video Demonstration Description:

The video runs the source code using `python src/main.py` and the program initializes. Training in the video only used 5 epochs and using a GPU for processing to keep the video short and still demonstrate the accuracy and loss improving over time. At end the output is shown, where the model is saved and the results from predicting on a portion of the original dataset that was set aside from training.

Results:

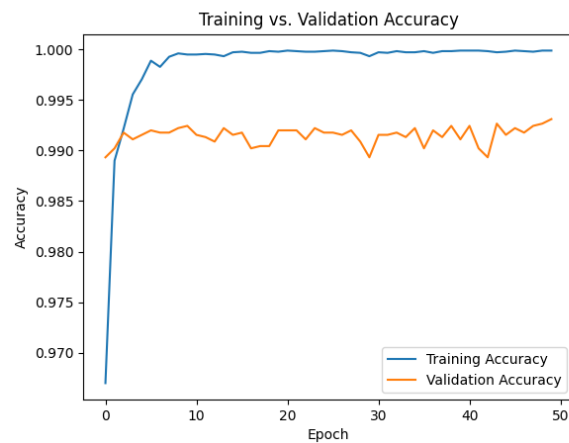
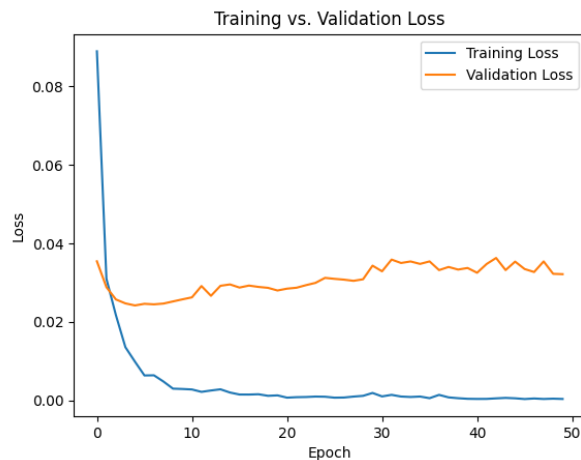
The final results received after runtime is:

Final Training Loss: 0.00039

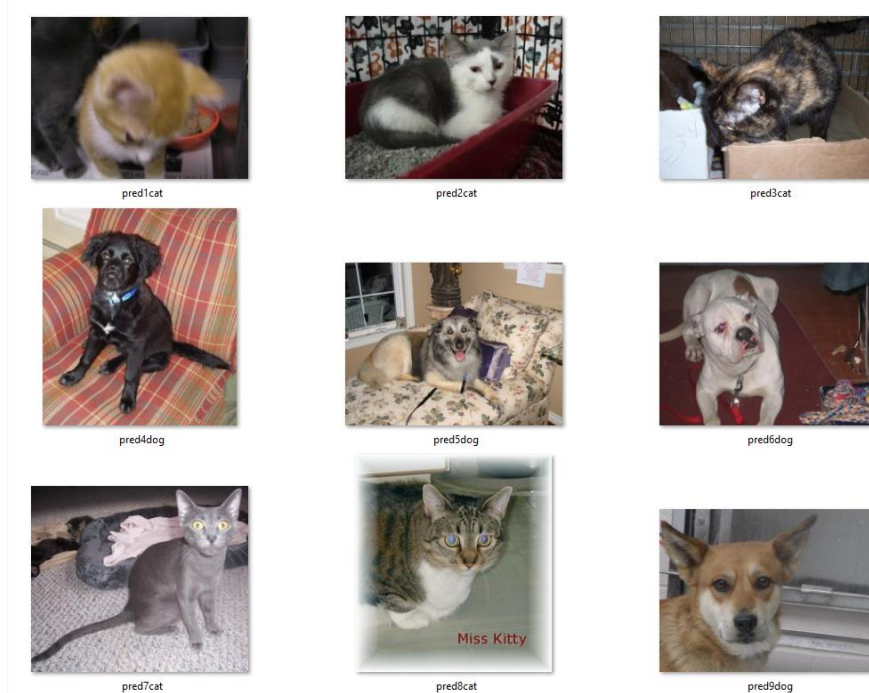
Final Training Accuracy: 99.988%

Final Validation Loss: 0.03219

Final Validation Accuracy: 99.31%



Examples of Images with predictions:



Challenges that happened throughout the development process include files in the dataset being corrupt, the dataset not being as outlined in instructions, and improvement of speed and performance.

Originally testing was done on a subset of the dataset to set up a pipeline and make sure run times were not long. Once pipeline was set up and testing began on the full dataset, errors occurred of unreferenceable images and null values when loading them in. To mitigate this problem two solutions were used. The first was to verify to image was not corrupt when loading initializing the ImageDataset. This was done using PIL `.verify()` to make sure the image had its integrity, without loading the full image into memory. The next solution was added during the `.__getitem__()` implementation of the ImageDataset object.

Using `.verify()` does not catch all corrupt images as it does not load the full image and check it. For any more images that could not be caught with this method a warning catcher was implemented to catch any warnings PIL threw while trying to read an image. A null value would be returned in place of the data. A collate function is then implemented in the data loader to skip any null values in each batch. This way all corrupt images are ignored and not used to train, validate or test.

The instructions included a description of what the dataset would look like. After setup of project and initial test, it was realized that the dataset was not as described in instructions. The solution for this was to analyze the dataset as to how it was laid out and what the file and directory names were and implement the necessary changes to the code to be able to handle correct format.

Matplotlib was used for image verification and confirm that the data pipeline was implemented correctly.

Implementing this project was done on a computer without a dedicated GPU, which meant processing power was limited and training had to be run on the CPU. This limited the speed at which running program had to be able to effectively test code. That led to the decision of using a subset originally for testing and using ResNet-18 as opposed to any other variants. ResNet-18 had the fastest speed and strong accuracy, although accuracy could have been improved upon if a different variant of ResNet had been used.

Potential Improvements:

Ways this project could be improved on in the future is including more threads, changing how images were loaded and improving the model being used.

More threading could be implemented for faster load times and processing. With the image loading in the dataset, where each image is checked to make sure that it is not corrupt, that process can take longer than needed. Implementing multiprocessing to load that faster would speed that up and make it more efficient, creating less waiting time and getting to training faster. It currently sits on a single main thread so implementing this would take creating threads and splitting the task among them. Increasing the number of workers to speed up training process could also have been implemented.

Improvement of the image formatting could have been done by converting them to more efficient formats. The code currently reads the .jpeg format every epoch. Converting images to tensors and storing them would have improved every epoch's fetch speed of data as it is much easier to read a tensor as opposed to an image, improving performance of training overall.

Final improvements that could be implemented in the future is the dataset and model. Using a larger dataset than the given should in turn return better results. Implementing random flips and rotation among other transformations also would have strengthened the model as it would have added more variance to the data and prevented overfitting. That would be a simple addition of just adding random flips and rotations when transforming the images when they are loaded in. Using a larger pretrained architecture would also improve on accuracy, while sacrificing time and memory usage. To use a different variant of ResNet could accomplish this but to determine which to use would depend on how much time and memory would be necessary to give up in comparison to improvements in accuracy.

Conclusion:

This computer vision project successfully implemented a complete image classification pipeline to identify between Cats and Dogs. Using a Convolutional Neural Network the model was able to correctly predict and identify images between the two categories with high accuracy and low loss. With a priority on efficiency, the model was still able to achieve high accuracy scores.